

DOCUMENTACION TECNICA

Sistema de Consulta de Pasajes

Next.js + TypeScript + SQL Server + integracion con ASP.NET/Crystal Reports

Objetivo del documento. Explicar de forma didactica y tecnica como funciona la aplicacion que estamos construyendo: autenticacion, sesiones, consulta de pasajes, integracion con SQL Server, visualizacion responsive y acceso controlado a los PDF del sistema existente.

Estado documentado: incluye autenticacion, consulta alfanumerica de documentos, PDF protegido, cierre de sesion y configuracion segura para GitHub

Contenido

1. Vision general del proyecto
2. Arquitectura general
3. Por que usamos Next.js + TypeScript
4. Estructura del proyecto
5. Cliente y servidor: la separacion fundamental
6. Flujo completo del inicio de sesion
7. Consulta del usuario en SQL Server
8. Compatibilidad con el cifrado del sistema C#
9. Que ocurre cuando las credenciales son correctas
10. Sesion, JWT y cookie HttpOnly
11. Proteccion del panel y uso del login
12. Consulta de pasajes
13. Validacion del documento
14. Ejecucion del SP BD_Pje_ConsultaPasajes
15. Transformacion de los resultados
16. Interfaz responsive: tabla y tarjetas
17. Proteccion de los parametros del PDF
18. Integracion con el sistema ASP.NET/Crystal Reports
19. Cierre de sesion
20. Variables de entorno y configuracion sensible
21. Responsabilidad de cada archivo
22. Consideraciones de seguridad
23. Flujo integral de extremo a extremo
24. Estado actual y siguientes pasos

1. Vision general del proyecto

Estamos construyendo una aplicacion web moderna que funciona como una capa de acceso segura y facil de usar sobre recursos que ya existen en _MGERENCIAL. La nueva aplicacion no reemplaza de inmediato la base de datos ni el generador de PDF anterior; reutiliza esos componentes y les agrega una interfaz moderna, responsive y controlada.

```

USUARIO (PC o celular)
  |
  v
NEXT.JS + TYPESCRIPT
- Interfaz React
- API / Backend
- Sesion y seguridad
  |
+-----> SQL SERVER (_MGERENCIAL)
  |
+-----> SISTEMA ASP.NET + CRYSTAL REPORTS

```

Idea principal. Next.js es el punto central. El navegador habla con Next.js; Next.js habla con SQL Server y con el sistema antiguo que genera los PDF.

2. Arquitectura general

La aplicacion tiene tres zonas principales: el navegador del usuario, el servidor Next.js y los sistemas existentes. Cada una cumple una responsabilidad distinta.

Navegador	Next.js	Sistemas existentes
Login	Valida credenciales	SQL Server
Formulario de documento	Ejecuta Stored Procedures	_MGERENCIAL
Tabla / tarjetas	Crea y verifica sesion	ASP.NET Web Forms
Boton Ver PDF	Protege parametros PDF	Crystal Reports

3. Por que usamos Next.js + TypeScript

En un proyecto tradicional podriamos separar React en un frontend y crear otro proyecto independiente para el backend. Para este sistema, Next.js nos permite mantener ambas partes juntas sin perder la separacion logica.

```

Proyecto unico Next.js
|
+-- React / interfaz
+-- TypeScript / tipado
+-- Route Handlers / API
+--Codigo servidor / SQL Server
+-- Control de sesiones

```

- Menos proyectos y menos configuracion para una aplicacion pequena.
- Podemos programar tanto la interfaz como la logica del servidor en TypeScript.

Proyecto Next.js + TypeScript + SQL Server

- Los Route Handlers permiten crear endpoints como /api/auth/login y /api/pasajes.
- Las credenciales de base de datos nunca necesitan llegar al navegador.

4. Estructura del proyecto

```
src/
|
+-- app/
|   +-- login/page.tsx
|   +-- panel/page.tsx
|   +-- panel/ConsultaPasajesClient.tsx
|   +-- api/auth/login/route.ts
|   +-- api/auth/logout/route.ts
|   +-- api/pasajes/route.ts
|   +-- documento/[token]/route.ts
|
+-- lib/
|   +-- db.ts
|   +-- session.ts
|   +-- encryption.ts
|   +-- pdfToken.ts
|
+-- types/
|   +-- usuario.ts
|   +-- pasaje.ts
```

La carpeta app contiene paginas y endpoints. La carpeta lib contiene servicios tecnicos reutilizables. La carpeta types contiene las estructuras TypeScript que describen los datos de usuarios y pasajes.

5. Cliente y servidor: la separacion fundamental

Esta separacion es uno de los conceptos mas importantes de todo el proyecto. Un componente que comienza con "use client" se ejecuta en el navegador. En cambio, los Route Handlers y los archivos de lib que usan credenciales de SQL deben ejecutarse en el servidor.

CLIENTE (navegador)	SERVIDOR (Next.js)
Mostrar formularios	Conectarse a SQL Server
Capturar numero de documento	Ejecutar Stored Procedures
Mostrar tabla/tarjetas	Desencriptar contraseña existente
Abrir un enlace PDF	Crear/verificar sesiones
Experiencia responsive	Conocer secretos y variables .env

Regla de seguridad. Nunca debemos colocar DB_PASSWORD, ENCRYPTION_KEY, JWT_SECRET o PDF_TOKEN_SECRET dentro de un componente cliente.

6. Flujo completo del inicio de sesion

1	El usuario escribe login y contraseña en /login.
2	React envia una peticion POST a /api/auth/login.
3	El servidor Next.js valida que existan ambos datos.
4	Next.js ejecuta __Pr_VerificaAccesoSistemaUsuario en SQL Server.
5	El SP devuelve los datos del usuario y la contraseña cifrada almacenada.
6	Next.js descripta esa contraseña con el algoritmo compatible con el sistema C#.
7	Se comparan la contraseña ingresada y la contraseña descriptada.
8	Si coinciden, se registra el ingreso en la bitacora.
9	Se crea una sesion y se guarda en una cookie HttpOnly.
10	El usuario es redirigido a /panel.

Importante. La contraseña descriptada se usa solo en el servidor durante la validacion. No se devuelve al navegador ni se guarda en la sesion.

7. Consulta del usuario en SQL Server

El procedimiento __Pr_VerificaAccesoSistemaUsuario recibe @login y devuelve la informacion necesaria para autenticar y contextualizar al usuario.

```
Datos relevantes devueltos:
- Id_Usuario
- login
- password
- estado
- Id_Trabajador
- Trabajador
- Sucursal
- IdPerfil
```

El usuario inactivo queda fuera de la consulta porque el SP filtra Usuario.estado = 1. En consecuencia, una cuenta inactiva se comporta como un usuario no encontrado para el flujo de login.

8. Compatibilidad con el cifrado del sistema C#

El sistema anterior no compara directamente el texto almacenado con la contraseña ingresada. Primero descripta el valor guardado. Para mantener compatibilidad con los usuarios existentes, el nuevo proyecto reproduce ese mismo proceso en TypeScript.

```
Sistema existente C#
  Base64 -> TripleDES (ECB, PKCS7)
          ^
          |
        clave MD5

Nuevo sistema TypeScript
  mismo algoritmo compatible
```

La finalidad actual no es rediseñar el esquema de contraseñas, sino permitir que las credenciales ya registradas sigan funcionando. En un sistema completamente nuevo seria preferible almacenar hashes no reversibles, por ejemplo con Argon2 o bcrypt.

Compatibilidad vs. diseño ideal. El cifrado reversible se conserva por compatibilidad con _MGERENCIAL. La aplicacion nueva limita su uso al servidor.

9. Que ocurre cuando las credenciales son correctas

Una vez que la contraseña coincide, el servidor realiza dos operaciones principales: registra el ingreso y crea la sesion.

1. Ejecuta pa_mantenimiento_Bitacora con datos como Usuario, Sucursal, Accion, Formulario, ModoIngreso y Host_Ip.
2. No muestra @pMensaje al usuario; solo puede usar @pRpta para saber si el registro fue correcto.
3. Crea un token de sesion con la identidad necesaria para las pantallas siguientes.
4. Devuelve una respuesta correcta y configura la cookie session.

10. Sesion, JWT y cookie HttpOnly

La sesion evita que el usuario tenga que volver a autenticarse en cada peticion. El token contiene identidad y contexto, no la contraseña.

```
Ejemplo conceptual del contenido:
{
  idUsuario: 35,
  login: "JPEREZ",
  trabajador: "JUAN PEREZ",
  sucursal: "CHICLAYO",
  idPerfil: 2
}
```

El token se guarda en la cookie session. Al ser HttpOnly, el JavaScript de la pagina no puede leerla mediante document.cookie. El navegador la envia automaticamente al servidor cuando corresponde.

- HttpOnly: reduce la exposicion del token al JavaScript del navegador.
- SameSite=Lax: limita ciertos envios de cookies entre sitios.
- Secure en produccion: la cookie se transmite por HTTPS.
- Expiracion: la sesion deja de ser valida despues del tiempo configurado.

11. Proteccion del panel y uso del login

La pagina /panel no confia simplemente en que el usuario llegue desde /login. Antes de mostrar la consulta verifica la cookie session y valida el token.

```
/panel
|
+-- ¿Existe cookie session?
|   +-- NO -> /login
|
+-- ¿Token valido?
    +-- NO -> /login
    +-- SI -> mostrar consulta
```

El login que se muestra en la cabecera proviene de la sesion ya creada. No hace falta volver a consultar SQL Server solo para mostrar ese dato.

12. Consulta de pasajes

La pantalla principal solicita un numero de documento y consulta los pasajes correspondientes. El documento puede ser numerico o alfanumerico, porque no siempre se trata de un DNI. El usuario controla solamente ese valor; el codigo de sucursal es una regla interna del sistema.

```

Usuario escribe numero de documento
  |
  v
GET /api/pasajes?dni=CE12345678
  |
  v
Servidor valida sesion + documento
  |
  v
BD_Pje_ConsultaPasajes
  |
  v
Resultados para la interfaz

```

Regla interna. CodigoSucursal se fija en 903 dentro de la logica del servidor; no se presenta como campo editable para el usuario.

13. Validacion del documento

El documento se valida en dos niveles. Tanto la interfaz como el backend aceptan letras y numeros, con una longitud de 8 a 20 caracteres. La interfaz mejora la experiencia del usuario y el backend protege realmente el endpoint.

Validacion en interfaz	Validacion en backend
El input admite texto en movil y escritorio.	No confia en el navegador.
Se permiten letras y numeros; se eliminan otros caracteres.	Acepta solo caracteres alfanumericos.
El boton se deshabilita con menos de 8 caracteres.	Exige entre 8 y 20 caracteres.
Da retroalimentacion inmediata.	Rechaza peticiones manipuladas manualmente.

Principio. La validacion del frontend es para experiencia de usuario. La validacion del backend es la que realmente protege el sistema.

14. Ejecucion del SP BD_Pje_ConsultaPasajes

El endpoint /api/pasajes obtiene una conexion desde el pool de SQL Server y ejecuta el Stored Procedure con parametros tipados.

```

Conceptualmente:
EXEC dbo.BD_Pje_ConsultaPasajes
    @CodigoSucursal = '903',
    @DNI = 'CE12345678'

```


El procedimiento devuelve como maximo 50 registros validos, desde la fecha actual en adelante y ordenados por FECHA_VIAJE. Entre los campos importantes estan ASIENTO, TIPO_DOC, SERIE, NUMERO, PASAJERO, ORIGEN, DESTINO, EMISION, PRECIO, FECHA_VIAJE y TURNO.

TIPO_DOC es necesario para generar el PDF, pero no se muestra como columna visible en la interfaz.

15. Transformacion de los resultados

La fila devuelta por SQL Server no se envia al navegador de manera ciega. El backend crea un objeto pensado especificamente para la interfaz.

```
SQL Server
ASIENTO
TIPO_DOC    -> uso interno
SERIE
NUMERO
PASAJERO
...

      |
      v

Objeto para la interfaz
asiento
serie
numero
emision
pasajero
origen
destino
precio
fechaViaje
turno
pdfUrl
```

Esta transformacion permite ocultar datos que no deben mostrarse y normalizar fechas, precios y nombres de propiedades.

16. Interfaz responsive: tabla y tarjetas

Los mismos resultados se presentan de forma diferente segun el ancho de pantalla.

```
ESCRITORIO
Asiento | Serie | Numero | Emision | Pasajero | Origen | Destino | Precio | Fecha viaje |
Turno | PDF

CELULAR
+-----+
| B001 - 123456 |
| JUAN PEREZ   Asiento 15|
| Chiclayo -> Lima |
| Fecha: 15/08/2026 |
| Turno: 03:00PM   |
| Precio: S/ 45.00 |
| [ Ver documento PDF ] |
+-----+
```

En movil evitamos obligar al usuario a desplazarse horizontalmente por una tabla muy ancha. Las tarjetas reorganizan la informacion manteniendo los mismos datos.

El turno se conserva tal como lo devuelve el SP, por ejemplo 03:00PM, sin recortarlo a 03:00.

17. Proteccion de los parametros del PDF

El sistema antiguo necesita Serie, Numero y TipoDocumento para generar el reporte. En lugar de exponer estos parametros directamente en el enlace visible, el backend genera un token cifrado temporal.

```
Datos internos:
Serie = B001
Numero = 123456
TipoDocumento = B

      |
      v
AES-256-GCM
      |
      v
/documento/<token-cifrado>
```

El token contiene tambien una fecha de expiracion. Al abrir el enlace, Next.js lo descifra, verifica que no haya expirado y recupera los parametros originales.

- El enlace visible no muestra Serie, Numero ni TipoDocumento.
- El token es temporal.
- El endpoint del documento valida la sesion.
- El navegador no necesita conocer la URL completa del sistema antiguo.

18. Integracion con el sistema ASP.NET/Crystal Reports

El endpoint /documento/[token] actua como intermediario. Despues de validar sesion y token, construye en el servidor la URL del sistema existente y solicita el documento.

```
Navegador
  |
  v
/documento/TOKEN
  |
  v
Next.js
- valida sesion
- descifra token
- obtiene Serie / Numero / TipoDocumento
  |
  v
ImpBD_PjeBoletaFactura.aspx
  |
  v
Crystal Reports
  |
  v
PDF
```

```

  |
  v
Next.js devuelve application/pdf
  |
  v
Navegador

```

El sistema ASP.NET recibe los parametros mediante QueryString. Segun TipoDocumento selecciona el reporte correspondiente, carga los datos, genera el PDF y redirecciona al archivo creado. Next.js sigue esa respuesta y devuelve el contenido al usuario.

Concepto tecnico. Next.js funciona aqui como una capa proxy/controlador: el usuario consume una URL del sistema nuevo y el servidor nuevo consume internamente el sistema antiguo.

19. Cierre de sesion

El boton Cerrar sesion vive en la misma cabecera de la pantalla de consulta. Al pulsarlo, la interfaz envia POST /api/auth/logout.

1	El usuario pulsa Cerrar sesion.
2	El servidor elimina la cookie session.
3	La interfaz reemplaza la pagina actual por /login.
4	Si el usuario intenta volver a /panel sin una nueva sesion, la pagina protegida lo redirige a /login.

20. Variables de entorno y configuracion sensible

Los datos sensibles se guardan en .env.local y se leen mediante process.env desde el servidor.

```

DB_SERVER=...
DB_DATABASE=_MGERENCIAL
DB_USER=...
DB_PASSWORD=...

JWT_SECRET=...
ENCRYPTION_KEY=...
PDF_TOKEN_SECRET=...
PDF_SERVICE_URL=...

```

.env.local no debe subirse a GitHub. Ademas de las credenciales y llaves, ahora tambien contiene PDF_SERVICE_URL para que la URL real del sistema ASP.NET no quede escrita en el codigo fuente publico. La variable debe llamarse PDF_SERVICE_URL y no NEXT_PUBLIC_PDF_SERVICE_URL, porque solo debe estar disponible en el servidor. Para documentar las variables necesarias sin revelar valores reales, se utiliza .env.example y se permite expresamente su versionado con !.env.example en .gitignore.

En Git, la regla `.env*` mantiene privados los archivos reales de entorno y la excepcion `!.env.example` permite subir solamente la plantilla vacia al repositorio.

Critico. `DB_PASSWORD`, `JWT_SECRET`, `ENCRYPTION_KEY` y `PDF_TOKEN_SECRET` deben permanecer fuera del repositorio.

21. Responsabilidad de cada archivo

Archivo	Responsabilidad
src/app/login/page.tsx	Pantalla del inicio de sesion.
src/app/api/auth/login/route.ts	Valida el usuario, registra bitacora y crea la sesion.
src/app/api/auth/logout/route.ts	Elimina la sesion.
src/app/panel/page.tsx	Protege la pagina y obtiene los datos del token de sesion.
src/app/panel/ConsultaPasajesClient.tsx	Formulario de documento, resultados, tabla, tarjetas y cierre de sesion.
src/app/api/pasajes/route.ts	Valida la peticion, ejecuta BD_Pje_ConsultaPasajes y prepara los resultados.
src/app/documento/[token]/route.ts	Valida y descifra el token, solicita y devuelve el PDF.
src/lib/db.ts	Configura y reutiliza el pool de conexiones a SQL Server.
src/lib/session.ts	Crea y verifica el token de sesion.
src/lib/encryption.ts	Replica el desenscriptado compatible con la aplicacion C# existente.
src/lib/pdfToken.ts	Cifra y descifra los parametros temporales del PDF.
src/types/usuario.ts	Define la estructura TypeScript del usuario devuelto por el SP.
src/types/pasaje.ts	Define la estructura que utiliza la interfaz para cada pasaje.
.env.local	Contiene valores reales de conexion, llaves y PDF_SERVICE_URL. No se versiona.
.env.example	Documenta las variables requeridas sin incluir valores sensibles. Si se versiona.
.gitignore	Ignora .env* y permite expresamente !.env.example para poder subir la plantilla.

22. Consideraciones de seguridad

- El navegador nunca se conecta directamente a SQL Server.
- Los secretos se mantienen en variables de entorno del servidor.
- La URL real del servicio PDF se obtiene desde PDF_SERVICE_URL en .env.local, por lo que no queda escrita en el codigo fuente del repositorio.
- Los Stored Procedures se ejecutan con parametros tipados, no concatenando SQL.
- La sesion se guarda en cookie HttpOnly.
- Cada endpoint sensible debe verificar la sesion de forma independiente.
- El numero de documento se valida tambien en el servidor y puede contener letras y numeros (8 a 20 caracteres).
- TIPO_DOC se usa internamente y no se presenta como columna al usuario.

- Los parametros del PDF se encapsulan en un token cifrado temporal.
- El acceso al documento sigue requiriendo una sesion valida.

Ocultar la URL del sistema anterior no sustituye una politica de seguridad en dicho sistema. La proteccion real del proyecto nuevo proviene de la autenticacion, validacion, control de sesiones, expiracion del token y ejecucion de logica sensible en el servidor.

23. Flujo integral de extremo a extremo

```

1. Usuario abre /login
2. Escribe login + contrasena
3. POST /api/auth/login
4. SQL: __Pr_VerificaAccesoSistemaUsuario
5. Desencriptado compatible con C#
6. Comparacion de contrasena
7. Bitacora
8. Cookie session
9. Redireccion a /panel
10. Usuario escribe numero de documento
11. GET /api/pasajes?dni=...
12. Validacion de sesion y documento
13. SQL: BD_Pje_ConsultaPasajes
14. Backend prepara resultados
15. Interfaz muestra tabla o tarjetas
16. Usuario pulsa Ver PDF
17. /documento/<token>
18. Validacion de sesion y token
19. Next.js consulta el ASP.NET existente
20. Crystal Reports genera el PDF
21. Next.js devuelve application/pdf
22. Navegador muestra el documento
23. Usuario pulsa Cerrar sesion
24. POST /api/auth/logout
25. Cookie eliminada y regreso a /login

```

24. Estado actual y siguientes pasos

Hasta este punto la arquitectura principal ya esta definida: autenticacion compatible con el sistema existente, sesion protegida, consulta de pasajes mediante Stored Procedure, documentos numericos o alfanumericos, interfaz responsive, enlace PDF protegido, URL del servicio PDF configurada fuera del codigo fuente y cierre de sesion.

- Probar el flujo completo en entorno local con datos reales.
- Revisar manejo de errores del sistema de PDF externo y tiempos de espera.
- Ajustar el diseno visual final con identidad corporativa si corresponde.
- Agregar registro adicional en bitacora para consultas o cierre de sesion si se necesita auditoria.
- Preparar configuracion de produccion con HTTPS y secretos propios del servidor.
- Revisar permisos y restricciones del repositorio antes del despliegue.

Resultado. La nueva aplicacion moderniza la experiencia sin obligar a reescribir de inmediato SQL Server, los Stored Procedures ni el generador de reportes existente.